Теория Параллелизма

Отчет

# Решение уравнения теплопроводности на двумерной сетке

Выполнил: 24943, Герман Даниил Фредерикович

26.06.2026

# Цель работы

Реализовать на C++ решение двумерного уравнения теплопроводности по явной разностной схеме (пятиточечный шаблон, итерации Якоби) на равномерных сетках 128x128, 256x256, 512x512, 1024x1024.

- Граничные условия — линейная интерполяция между угловыми значениями (10, 20, 30, 20 по обходу), внутренняя область инициализируется нулями (тип элементов double).
- Параметры (точность, размер сетки, максимум итераций, частота проверки ошибки) задаются через командную строку (boost::program_options).
- Перенести расчёт на GPU директивами OpenACC, сравнить производительность вариантов -acc=host, -acc=multicore, -acc=gpu для разных размеров сеток, выполнить профилирование в Nsight Systems и оптимизировать код.

## Используемый компилятор

- nvc++ (NVIDIA HPC SDK 23.11) с ключами -acc=<host|multicore|gpu> и -Minfo=all.
- Сборка через CMake (CMakeLists.txt), для каждого режима OpenACC собирается отдельная цель: heat_eq_host, heat_eq_multicore, heat_eq_gpu.
- NVTX-метки включаются макросом USE_NVTX при наличии CUDA Toolkit.

## Используемый профилировщик

NVIDIA Nsight Systems (nsys). Для разбивки по этапам в код добавлены NVTX-метки solve / calcNext / swap; средние времена этапов извлекались отчётом nsys stats --report nvtx_pushpop_sum.

## Как производили замер времени работы

Замер полного времени счёта выполняется библиотекой <chrono> (std::chrono::steady_clock): таймер ставится непосредственно перед блоком #pragma acc data (т.е. включает копирование данных на устройство и обратно) и снимается после #pragma acc update host. Результат выводится в секундах вместе с числом итераций и достигнутой ошибкой.

# Выполнение на CPU

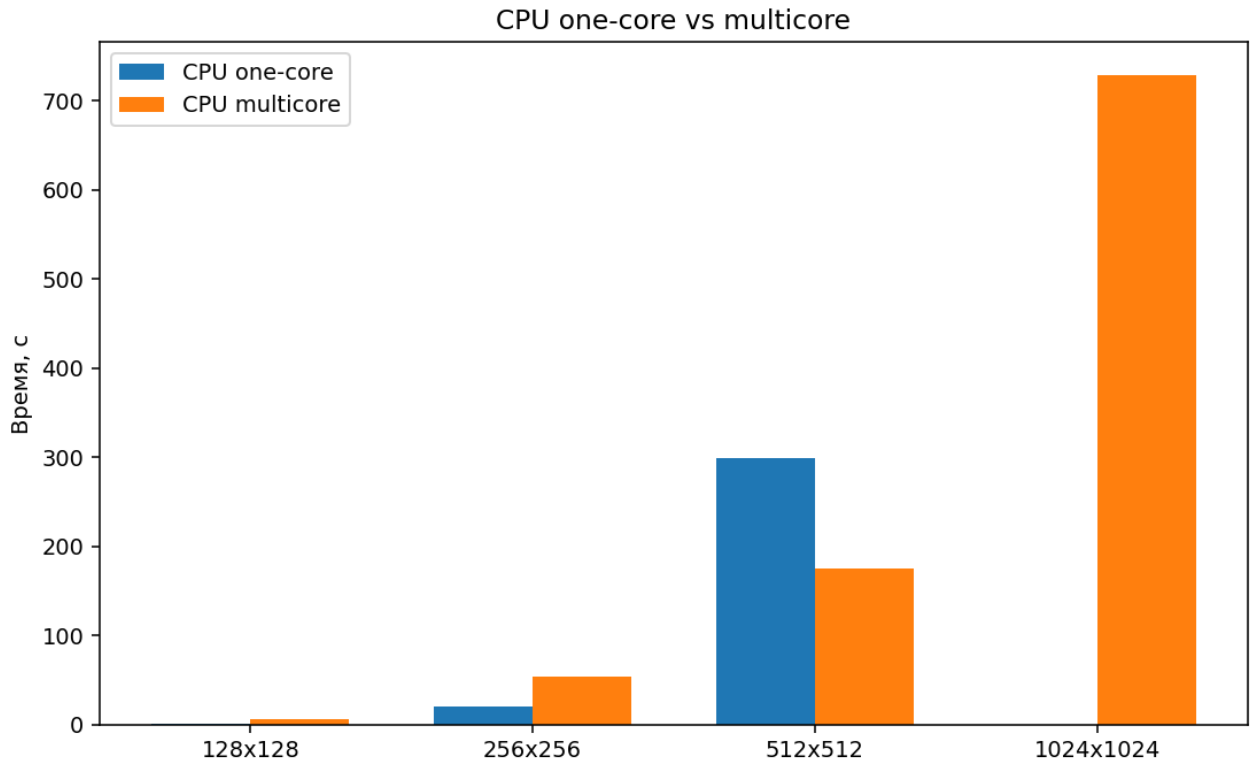
CPU-onecore

| Размер Сетки | Время выполнения (с) | Точность | Количество итераций |
|---|---|---|---|
| 128*128 | 1,436 | 9,999803E-07 | 30 074 |
| 256*256 | 20,886 | 9,999302E-07 | 102 885 |
| 512*512 | 304,359 | 9,999842E-07 | 339 599 |

CPU-multicore

| Размер Сетки | Время выполнения (с) | Точность | Количество итераций |
|---|---|---|---|
| 128*128 | 8,873 | 9,999803E-07 | 30 074 |
| 256*256 | 165,825 | 9,999302E-07 | 102 885 |
| 512*512 | 244,424 | 9,999842E-07 | 339 599 |
| 1024*1024 | 554,952 | 1,369286E-06 | 1 000 000 |

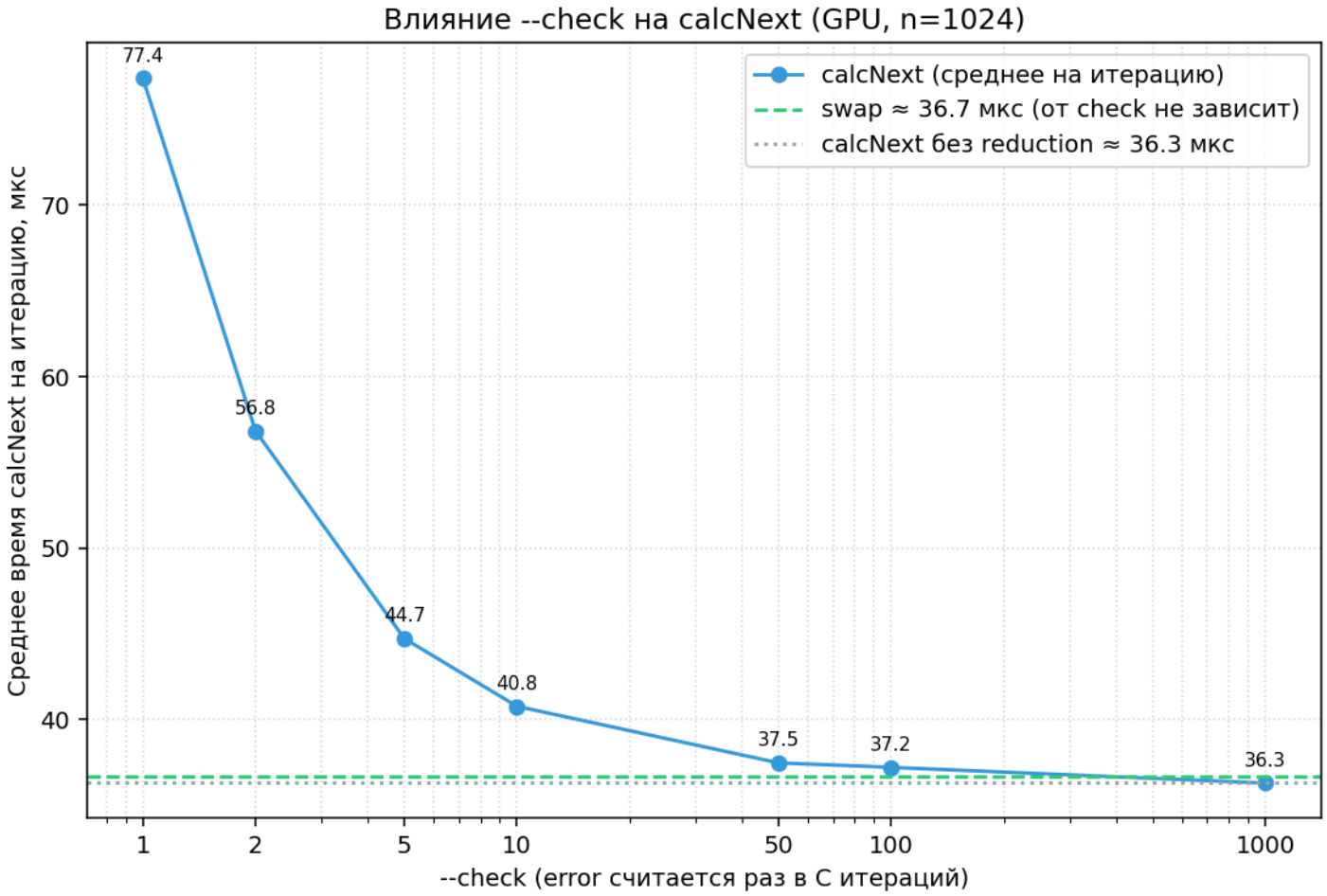
Диаграмма сравнения время работы CPU-one и CPU-multi

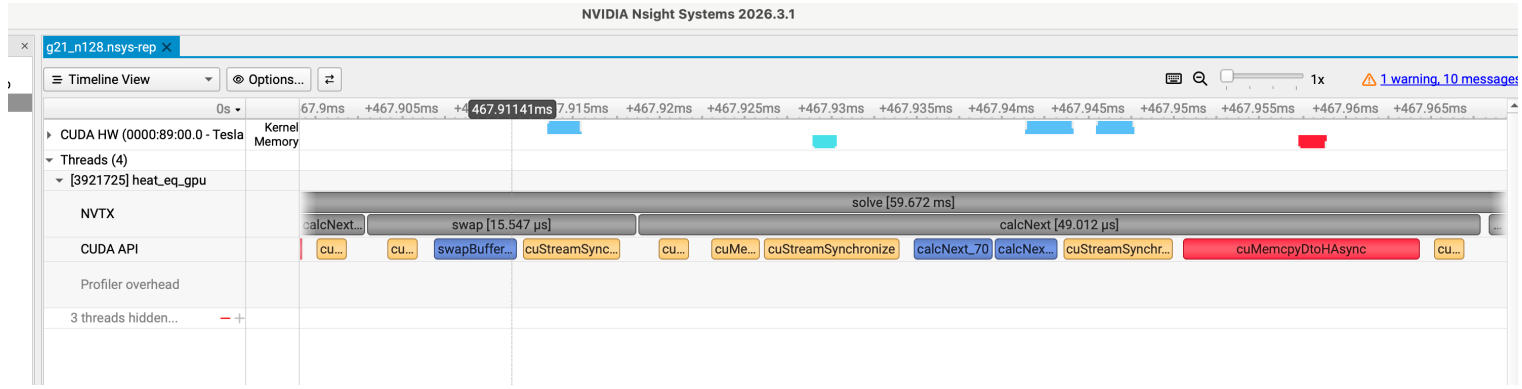


# Выполнение на GPU

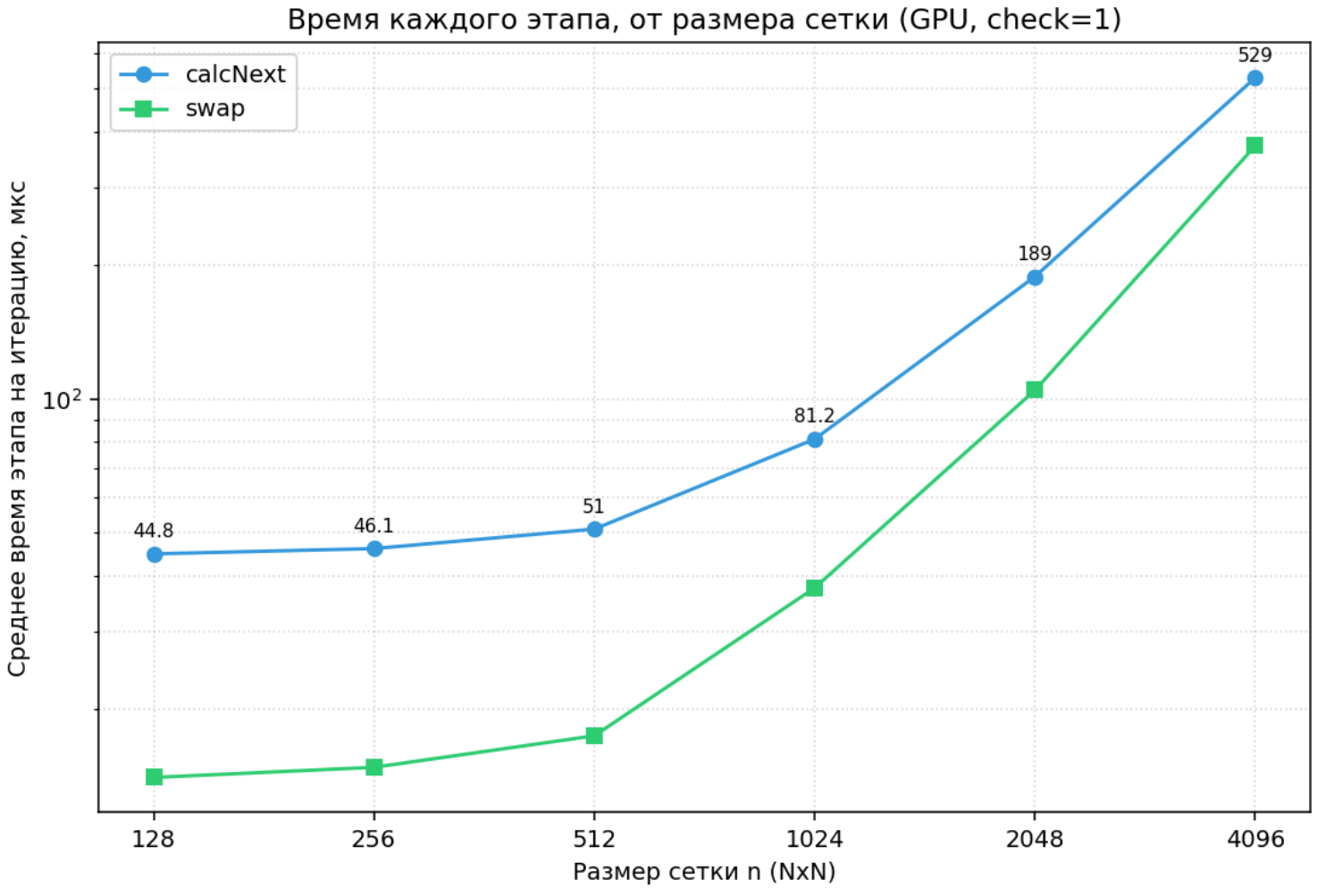
Этапы оптимизации на сетке 512*512

| Этап № | Время выполнения (с) | Точность | Максимальное выполнения количество | Комментарии (что было итераций сделано) |
|---|---|---|---|---|
| 1 | 96,564 | 1,369286E-06 | 1 000 000 | Базовый перенос на GPU: данные резидентны на устройстве (acc data copyin + present),<br>но reduction(max:error) считается каждую итерацию (check = 1) |
| 2 | 66,001 | 1,369926E-06 | 1 000 000 | Ошибка вычисляется раз в 100 итераций (check=100). Т.е. дорогая редукция убрана из большинства итераций; ускорение 32% |
| 3 | 65,811 | 1,375752E-06 | 1 000 000 | check=1000. Уменьшение чек в 10 раз не привело к таким же изменениям как скачок с check=1 к check=100. |




NVIDIA Nsight Systems 2026.3.1
g21_n128.nsys-rep
Timeline View
Options...
1x
1 warning, 10 messages
CUDA HW (0000:89:00.0 - Tesla
Kernel Memory
Threads (4)
[3921725] heat_eq_gpu
NVTX
solve [59.672 ms]
calcNext...
swap [15.547 µs]
calcNext [49.012 µs]
CUDA API
swapBuffer...
cuStreamSync...
cuStreamSynchronize
calcNext_70
cuStreamSynchr...
cuMemcpyDtoHAsync
Profiler overhead
3 threads hidden...

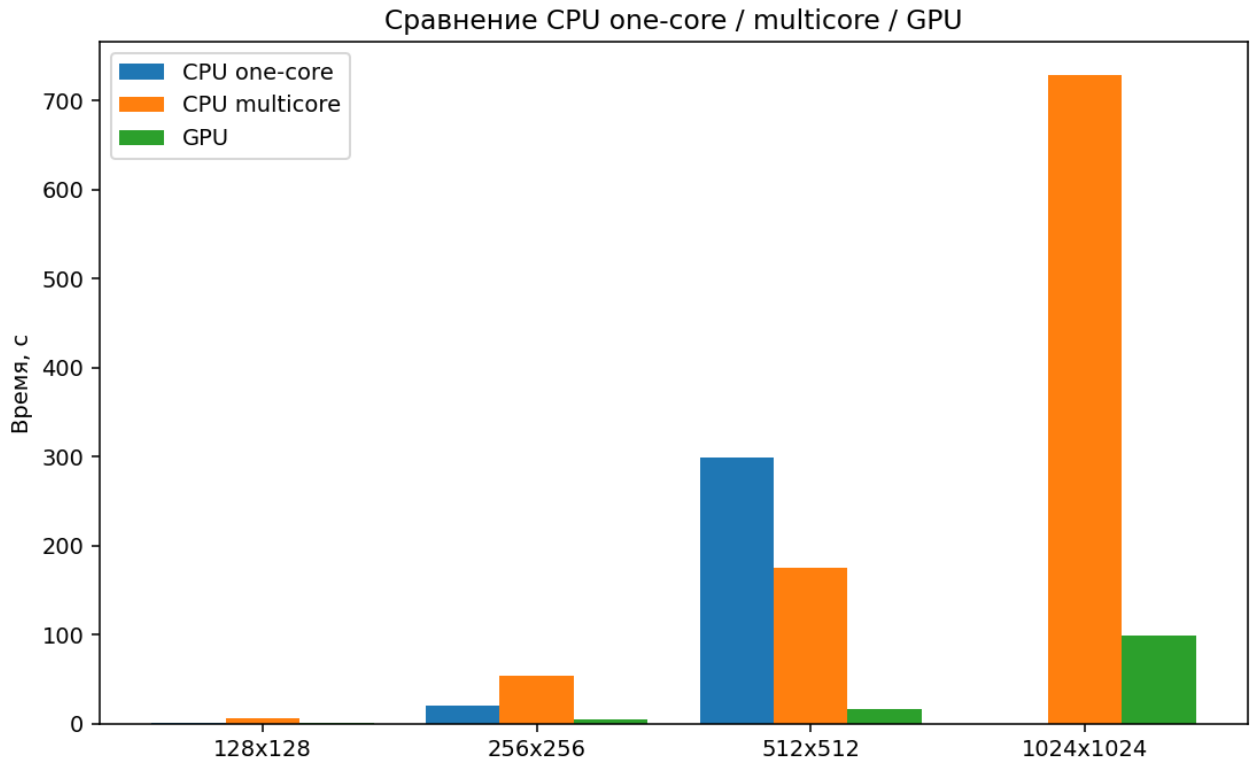

NVIDIA Nsight Systems 2026.3.1
g21_n128.nsys-rep
Timeline View
Options...
1x
1 warning, 10 messages
CUDA HW (0000:89:00.0 - Tesla
Kernel Memory
Threads (4)
[3921725] heat_eq_gpu
NVTX
solve [59.672 ms]
CUDA API
Profiler overhead
3 threads hidden...


Время каждого этапа, от размера сетки (GPU, check=1)
calcNext
swap
Среднее время этапа на итерацию, мкс
$10^2$
44.8
46.1
51
81.2
189
529
128
256
512
1024
2048
4096
Размер сетки n (NxN)

# GPU — оптимизированный вариант

| Размер сетки | Время выполнения (с) | Точность | Количество итераций |
|---|---|---|---|
| 128*128 | 1,363 | 9,999803E-07 | 30 074 |
| 256*256 | 4,529 | 9,999302E-07 | 102 885 |
| 512*512 | 16,777 | 9,999842E-07 | 339 599 |
| 1024*1024 | 96,564 | 1,369286E-06 | 1 000 000 |

## Диаграмма сравнения времени работы CPU-one, CPU-multi, GPU(оптимизированный вариант) для разных размеров сеток



# Вывод

В работе реализована и распараллелена явная схема Якоби для уравнения теплопроводности; корректность подтверждена выводом сеток 10×10 и 13×13 (граница — линейная интерполяция углов 10/20/30/20, внутренность сходится к гладкому полю).

**Ответы на вопросы анализа:**

1. **Что ограничивает производительность?** На GPU итерация состоит из двух проходов по всей сетке (calcNext и swap) — задача “memory-bound”, время растёт линейно с числом узлов.
   Дополнительный тормоз это reduction(max:error) каждую итерацию: она почти удваивала стоимость calcNext (77 мкс против 37 мкс на 1024x1024). На CPU-multicore на малых сетках всё съедают накладные расходы на параллелизм.
2. **Как исправить?**
   (1) Вычислять ошибку не каждую итерацию, а раз в check итераций — это дало 32 % ускорения на GPU (66,0с против 96,6с) практически без потери точности.
   (2) Держать данные на вычисляемом устройстве (acc data, present) (без копирование host<->device на каждом цикле).
   (3) swap целесообразно заменить “пинг-понгом” указателей вместо физического копирования Anew —> A — это убирает целое ядро (30–40 % времени итерации на больших сетках).
   (4) Для реального ускорения сходимости — перейти от Якоби к методу с лучшей сходимостью (Гаусса–Зейделя/красно-чёрный, SOR), чтобы 1024x1024 сходилось за <1e6 итераций.
3. **Делает ли программа что-то лишнее?** Да: в базовом варианте — лишняя редукция максимума на каждой итерации (исправлено параметром check) и отдельное ядро копирования swap (устранимо ping-pong-ом буферов). Эти два пункта и есть основной резерв оптимизации.

Итог по ускорению (относительно CPU-onecore): GPU даёт ×4.6 на 256x256 и ×18 на 512x512; чем крупнее сетка, тем выгоднее GPU. CPU-multicore оправдан только на сетках от 512x512 и заметно уступает GPU на всех размерах.